WATER TREATMENT IOT MONITORING PROJECT

1. Purpose and scope

This repository is a locally runnable demonstration of an Internet of Things (IoT) monitoring stack for a simulated water-treatment process. It creates artificial sensor readings, validates them, stores them in SQLite, exposes them through a FastAPI service, and displays selected measurements in a Streamlit dashboard.

The project is for learning and portfolio demonstration. It is not connected to a real water-treatment plant. It monitors and displays data only; it does not operate pumps, valves, chemical dosing, or any other equipment.

2. What the application does

The simulated process produces readings for a timestamp, plant identifier, inlet and outlet flow, tank level, inlet and outlet pressure, pH, turbidity, pump running status, and pump power. Values are generated in Python, rather than collected from real sensors. The simulator also inserts occasional artificial events so the alarm display can be demonstrated.

A reading can be submitted to the API over HTTP. The API validates the payload before it is stored. Data is retained in a local SQLite database. The API can return the stored readings, calculate summary averages, and evaluate illustrative alarms for the latest reading.

The dashboard requests summary values and recent telemetry from the API. It displays record count and KPI cards, trends for flows, tank level, pH, turbidity and pump power, the latest illustrative alarms, and a table of recent readings.

3. Main parts of the project

The water_monitoring/simulator.py module generates repeatable synthetic readings. A seed and reading index make individual generated readings reproducible. The generated series includes occasional artificial pH, turbidity, tank-level, and flow events.

The water_monitoring/schemas.py module defines Pydantic models for input and output. It checks expected data types and reasonable demo bounds, and rejects unexpected extra fields. These limits are input-validation bounds for this software demonstration, not treatment standards.

The water_monitoring/service.py module connects ingestion with storage, alarm evaluation, and the specific-energy calculation.

The water_monitoring/storage.py module creates and uses the SQLite telemetry table. It supports inserting a reading, listing recent readings, calculating summaries, and clearing stored telemetry. A timestamp index supports ordering readings by time.

The water_monitoring/alarms.py module evaluates example conditions for pH, turbidity, tank level, pressure, flow mismatch, and a pump status/power mismatch. The messages and thresholds are explanatory demonstration logic only.

The water_monitoring/energy.py module calculates indicative specific energy as pump power

divided by outlet flow. The resulting units are kWh per cubic metre. If flow is zero, the result is unavailable rather than a division by zero.

The water_monitoring/api.py module provides the FastAPI application and HTTP routes. The water_monitoring/dashboard.py module provides the Streamlit user interface. The water_monitoring/cli.py module can seed the database with generated sample readings.

The optional water_monitoring/mqtt_publisher.py and water_monitoring/mqtt_worker.py modules demonstrate publishing synthetic readings and storing MQTT messages. MQTT use requires a running broker. Dockerfile and docker-compose.yml describe a container-based API and dashboard setup.

4. Data flow

First, the simulator or an external caller creates a telemetry reading. For HTTP ingestion, the reading is sent to the API, validated, passed to the service, and written to SQLite. The API can then return stored readings, summary KPIs, or alarms. The dashboard reads those API responses and turns them into cards, charts, alarm messages, and a table.

The optional MQTT path sends JSON messages through a broker to a worker. The worker validates incoming data with the telemetry schema and writes accepted readings to SQLite.
The MQTT path is optional and separate from the basic local HTTP demonstration.

5. HTTP API overview

GET /health reports that the API is available and identifies its mode as simulation-only.

POST /telemetry accepts one telemetry reading, validates it, and stores it. The response includes the stored identifier, any alarms for that reading, and its indicative specific-energy value.

GET /telemetry?limit=200 returns recent readings. The limit defaults to 200 and is bounded by the API to a maximum of 5000.

GET /summary returns record count, latest timestamp, and averages for available flow, pH, turbidity, and specific-energy data. Some averages can be null when there is no corresponding data.

GET /alarms/latest evaluates alarms for the most recently stored reading. If no reading exists, it returns an empty alarm list.

The FastAPI interactive documentation is normally available at http://127.0.0.1:8000/docs while the API is running.

6. Dashboard error and correction

The reported exception was: AttributeError: NoneType object has no attribute json. It occurred in water_monitoring/dashboard.py on the line that assigned the result of raise_for_status() to summary and then attempted summary.json().

The root cause is that requests.Response.raise_for_status() checks the HTTP status and raises an exception for an unsuccessful response, but returns None when the response is successful. Therefore, assigning its return value to summary replaced the HTTP response with None. Calling .json() on that value caused the AttributeError.

The dashboard now keeps the HTTP response in summary_response, calls summary_response.raise_for_status() as a separate status check, and then reads the response body with summary_response.json(). This preserves the response object for JSON decoding while retaining the existing HTTP error handling. The telemetry request follows the same separate request, status check, and JSON-decoding pattern.

If the API is unreachable or returns an HTTP error, the dashboard's request handling displays an error message and stops rendering the dependent dashboard content. The API must be started and available at the configured address for data to load.

7. Running the local demonstration

Use Python 3.10 or newer. From the project root, create and activate a virtual environment and install requirements:

```
python -m venv .venv     .venv\Scripts\Activate.ps1     pip install -r requirements.txt
```

Seed the database with 200 synthetic readings:

```
python -m water_monitoring.cli seed --count 200 --clear
```

Start the API in one terminal:

```
uvicorn water_monitoring.api:app --reload Start the dashboard in another terminal:
```

```
streamlit run water_monitoring/dashboard.py
```

The dashboard defaults to API address http://127.0.0.1:8000. WATER_API_URL can be set to use a different API address. WATER_DB_PATH can be set to change the database file location. If no readings have been seeded, the dashboard explains how to add demo telemetry.

The project also includes a Dockerfile and Docker Compose configuration for running an API and dashboard in containers. The Compose dashboard is configured to contact the API by its service name. See README.md for the full setup instructions and optional MQTT commands.

8. Tests and verification status

The tests/test_core.py file includes checks for the specific-energy calculation, deterministic and bounded simulator values, rejection of a zero reading count, a high-turbidity alarm, storage and service ingestion, and an API end-to-end flow.

The test suite was not successfully run during this documentation task. VS Code test discovery did not match the requested test file, and a direct attempt through the selected workspace Python environment stopped because pytest is not installed in that environment. The project lists pytest in requirements.txt; install the project dependencies in the intended environment and run pytest -q to execute the suite. This report does not claim that the tests passed.

9. Important limitations

All telemetry described here is synthetic. Alarm thresholds are illustrative examples, not regulatory limits, engineering setpoints, or safety protections. The energy number is an indicative ratio, not an audited efficiency measure. The dashboard is read-only and the application contains no process-control loop.

The software has not been validated for real plant operation, regulatory compliance, safety, instrument calibration, or operational reliability. Do not connect it to control equipment or use its alarms to make real treatment decisions. A real deployment would require qualified process engineering, security and safety reviews, reliable calibrated instrumentation, operational testing, and approval by the site owner.

10. Summary of work recorded

The project provides a simulated telemetry-to-dashboard demonstration using Python, Pydantic, SQLite, FastAPI, and Streamlit, with optional MQTT and Docker examples. The specific issue addressed in this work was the dashboard's NoneType error when parsing the summary response. The code now checks the HTTP response status without replacing the response object, then decodes its JSON data. Automated test execution remains unverified because pytest is absent from the selected Python environment.